

# Static Class in C# with Examples

In this article, I am going to discuss **Static Class in C#** with Examples. Please read our previous article where we discussed [Extension Methods in C#](#) with Examples. I am sure at the end of this article; you will understand the need and use for Static Class in C# with Examples.

## Static Class in C#

The class which is created by using the static modifier is called a static class in C#. A static class can contain only static members. It is not possible to create an instance of a static class. This is because it contains only static members. And we know we can access the static members of a class by using the class name.

## Example to understand the Static Class in C#:

Let us understand the need and use of Static Class in C# with Example. First, create a console application with the name StaticClassDemo.

### CountryMaster.cs:

Once you created the Console application, then add a class file with the name **CountryMaster.cs** and then copy and paste the below code into it. Here we have created the class with three properties and one method. The CountryCode property is going to hold the three-letter symbols of the country while the CountryName property going to hold the full country name. The ComputerName property has the logic to retrieve the current machine name. The Insert Method inserts the country record into the database and while inserting it also uses the ComputerName property to tell that from which computer this record was inserted.

```
namespace StaticClassDemo
{
    public class CountryMaster
    {
        public string CountryCode { get; set; }
        public string CountryName { get; set; }
        private string ComputerName
        {
            get
            {
                return System.Environment.MachineName;
            }
        }
        public void Insert()
        {
            //Insert the data
        }
    }
}
```

### Customer.cs

Next, create another class file with the name **Customer.cs** and then copy and paste the following code into it.

```
namespace StaticClassDemo
{
    public class Customer
    {
        public string CustomerCode { get; set; }
        public string CustomerName { get; set; }
        private string MachineName = "";
        private bool IsEmpty(string value)
        {
            if (value.Length > 0)
            {
                return true;
            }
            return false;
        }
        public void Insert()
        {
            if (IsEmpty(CustomerCode) && IsEmpty(CustomerName))
            {
                //Insert the data
            }
        }
    }
}
```

### Explanation of Above Code:

The **CustomerCode** property is going to hold the three-letter code of the customer while the **CustomerName** property holds the customer's name. The **IsEmpty** method accepts one value and then checks if the value is empty or not. If not empty then return true else return false. The **Insert** method simply checks if both **CustomerCode** and **CustomerName** are not empty then insert the customer record into the database.

Here, the problem is with the **MachineName** variable. The **MachineName** should have the current computer name while inserting the customer data into the database so that we can track from which machine this customer data was inserted

If you remember, the **CountryMaster** class has the logic to retrieve the computer name. Rather than writing the duplicate logic here, we should go and use the logic which is already written in the **CountryMaster** class, so that we can avoid writing repeating code or redundant code.

If you check the **ComputerName** property in the class **CountryMaster.cs** file, then you will see that, it is private, so in order to use that property in the **Customer** class, first, we need to change it to the public as shown in the below image.

```

public class CountryMaster
{
    public string CountryCode { get; set; }
    public string CountryName { get; set; }
    public string ComputerName
    {
        get
        {
            return System.Environment.MachineName;
        }
    }
    public void Insert()...
}

```

Again, while inserting the CountryMaster record into the database, we also need to check both CountryCode and CountryName should not be empty. To check if empty or not, we also like to use the **IsEmpty** method which is defined in the Customer class rather than writing the complete logic here. Further, if you notice, the IsEmpty method of the Customer class is private, so in order to use that method in CountryMaster class, we need to change it to the public as shown in the below image.

```

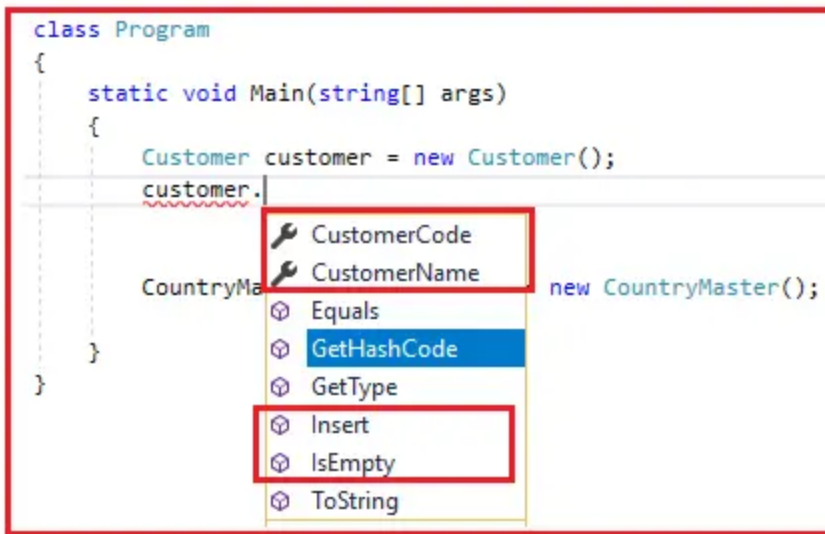
public class Customer
{
    public string CustomerCode { get; set; }
    public string CustomerName { get; set; }
    private string MachineName = "";
    public bool IsEmpty(string value)
    {
        if(value.Length > 0)
        {
            return true;
        }
        return false;
    }
    public void Insert()...
}

```

The CountryMaster class has logic to retrieve the computer name and we want to use that logic in the Customer class so we made the ComputerName property public. Similarly, the Customer class has the logic check whether a value is empty or not and we also want that logic in the CountryMaster class, so we made the IsEmpty method as public. As long as we did this, we violate the [encapsulation principle](#).

## How we are Violating the OOPs Encapsulation Principle?

Let us understand how we are violating the encapsulation principle. Modify the Program class as shown below. Once you created the Customer class object, then you can see the public member of that class as shown in the below image.



As you can see, we have exposed the CustomerCode, CustomerName, Insert, and IsEmpty methods. There is a clear violation of abstraction. Abstraction means showing only what is necessary. So, the external person who is consuming your class, should see and consume the **CustomerCode**, **CustomerName**, and **Insert** method. But should not see the **IsEmpty** method. The **IsEmpty** method is for internal use i.e. use by other methods not by the consumer of the class. As we make the IsEmpty method as public, we are violating the Encapsulation principle.

In the same way, we also violating the abstraction principle with the **CountryMaster** object as we are exposing the **ComputerName** property to the external world that is going to consume the class as shown in the below image. The **ComputerName** property is for internal use only.



**Note:** With the above, we are achieving code reusability (reusing the ComputerName and IsEmpty method) but violating the encapsulation principle.

## How to solve this problem?

How to solve the above problem means how should we achieve code reusability without violating the OOPs principles (i.e. Encapsulation Principle). In order to achieve both, let us add a new class and then move those two functions into that class. Create a class file with the name **CommonTask.cs** and then copy and paste the following code into it.

```

namespace StaticClassDemo
{
    public class CommonTask
    {
        public bool IsEmpty(string value)
        {
            if (value.Length > 0)
            {
                return true;
            }
            return false;
        }
        public string GetComputerName()
        {
            return System.Environment.MachineName;
        }
    }
}

```

Please remove the IsEmpty() method from the Customer class and the ComputerName property from the CountryMaster class. Now both the logic which violates the OOPs principle has been moved to the **CommonTask** class.

## Modifying Customer class:

Now modify the Customer class as shown below. As you can see, in the constructor we set the value of the MachineName private variable and in the Insert method, we create an instance of **CommonTask** class and Invoke the **IsEmpty** method.

```

namespace StaticClassDemo
{
    public class Customer
    {
        public string CustomerCode { get; set; }
        public string CustomerName { get; set; }
        private string MachineName = "";
        public Customer()
        {
            CommonTask commonTask = new CommonTask();
            MachineName = commonTask.GetComputerName();
        }
        public void Insert()
        {
            CommonTask commonTask = new CommonTask();
            if (!commonTask.IsEmpty(CustomerCode) && !commonTask.IsEmpty(Custc
            {
                //Insert the data
            }
        }
    }
}

```

```
}
```

## Modifying the CountryMaster class:

Please modify the **CountryMaster** class as shown below. Here, we created the instance of **CommonTask** and then Invoke the **GetComputerName** and **IsEmpty** methods.

```
namespace StaticClassDemo
{
    public class CountryMaster
    {
        public string CountryCode { get; set; }
        public string CountryName { get; set; }
        private string ComputerName
        {
            get
            {
                CommonTask commonTask = new CommonTask();
                return commonTask.GetComputerName();
            }
        }
        public void Insert()
        {
            CommonTask commonTask = new CommonTask();
            if (!commonTask.IsEmpty(CountryCode) && !commonTask.IsEmpty(CountryName))
            {
                //Insert the data
            }
        }
    }
}
```

As we centralized the **IsEmpty** and **GetComputerName** methods in the **CommonTask** class, we can use these methods in both the **Customer** and **CountryMaster** classes. The above solution seems to be decent as it does not violate the **OOPs Principle** and also achieves code reusability and I hope many of you also agree to it. But there is also some problem.

## What is the problem in the above solution?

In order to understand the problem, let us first analyze the **CommonTask** class in a great manner.

1. This **CommonTask** class is a collection of unrelated methods and properties that are not related to each other. Because it has unrelated methods, properties, or logic, it does not represent any real-world objects.
2. As it does not represent any real-world objects, so any kind of OOPs principles (inheritance, abstraction, polymorphism, encapsulation) should not be allowed to be applied to this **CommonTask** class.
3. So, in simple words, we can say that this is a fixed class i.e. a class with a fixed behavior. That is, its behavior can not be changed by inheritance, and its behavior can not be polymorphed by using either static or dynamic polymorphism. So, we can say that this class is a fixed class or static class.

## How do we avoid Inheritance, how do we avoid abstract keywords, or how do we avoid the OOPs principle in a class?

The answer is by using the **static** keyword. So, you need to mark the **CommonTask** class as **static** by using the static keyword. When you mark a class as **static**, everything inside the class should be static. That means, we also need to mark the **IsEmpty** and **GetComputerName** methods as static. So, modify the **CommonTask** class as shown below.

```
namespace StaticClassDemo
{
    public static class CommonTask
    {
        public static bool IsEmpty(string value)
        {
            if (value.Length > 0)
            {
                return true;
            }
            return false;
        }
        public static string GetComputerName()
        {
            return System.Environment.MachineName;
        }
    }
}
```

Once you make the class static, then you cannot use the **new** keyword with the static class to create an instance, rather you need to invoke the **IsEmpty** and **GetComputerName** methods by using the class name. Internally only one instance of the static class gets created by CLR which serves all the clients.

### Modify the Customer class:

Now modify the Customer class as shown below. As you can see, now we are invoking the **GetComputerName** and **IsEmpty** method using the class name i.e. **CommonTask**.

```
namespace StaticClassDemo
{
    public class Customer
    {
        public string CustomerCode { get; set; }
        public string CustomerName { get; set; }
        private string MachineName = "";
        public Customer()
        {
            MachineName = CommonTask.GetComputerName();
        }
        public void Insert()
        {

```

```

        if (!CommonTask.IsEmpty(CustomerCode) && !CommonTask.IsEmpty(Custc
        {
            //Insert the data
        }
    }
}

```

## Modify the CountryMaster Class:

Modify the **CountryMaster** class as shown below. As you can see in the below code, we are invoking the **GetComputerName** and **IsEmpty** method using the class name i.e. **CommonTask**.

```

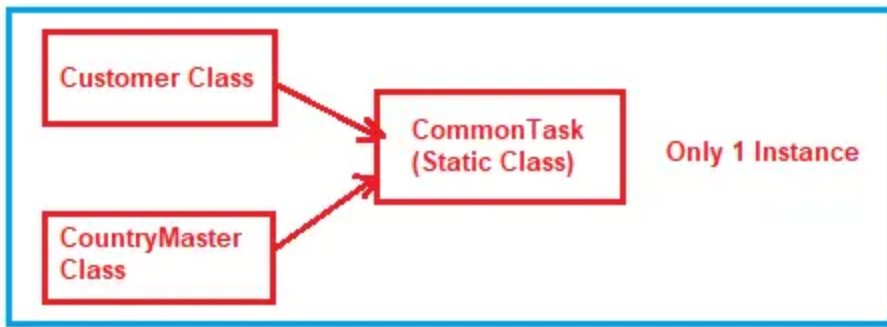
namespace StaticClassDemo
{
    public class CountryMaster
    {
        public string CountryCode { get; set; }
        public string CountryName { get; set; }
        private string ComputerName
        {
            get
            {
                return CommonTask.GetComputerName();
            }
        }
        public void Insert()
        {
            if (!CommonTask.IsEmpty(CountryCode) && !CommonTask.IsEmpty(Countri
            {
                //Insert the data
            }
        }
    }
}

```

## How is the static Class Instantiated in C#?

We cannot apply any OOPs principles to the static class like inheritance, polymorphism, encapsulation, and abstraction. But in the end, it is a class. And at least to use a class it has to be instantiated. If the static class is not instantiated then we cannot invoke the methods and properties that are present in the static class. Now let us see how the instantiation takes place internally of a static class i.e. in our example, it is the **CommonTask** class.

The CLR (Common Language Runtime) will create only one instance of the **CommonTask** class irrespective of how many times they called from the **Customer** and **CountryMaster** class. For a better understanding, please have a look at the below image.



Due to the single instance behavior, the static class is also going to be used to share the common data.

## Can we create an Instance of a Static Class in C#?

We cannot create an instance of a static class in C#. For a better understanding, please have a look at the below code. In our example, CommonTask is a static class and hence we cannot create an instance of the CommonTask, and if we try then we will get a compile-time error as shown in the below image.

```
class Program
{
    static void Main(string[] args)
    {
        CommonTask obj = new CommonTask();
        Console.ReadKey();
    }
}
```

 `class StaticClassDemo.CommonTask`  
Cannot create an instance of the static class 'CommonTask'

## Can we Inherit a Static Class in C#?

Internally, static classes are sealed, which means we cannot inherit a static class from another class. For a better understanding, please have a look at the below image. Here, we are trying to inherit the static class and hence we are getting a compile-time error.

```
public static class CommonTask
{
    public static bool IsEmpty(string value) {...}
    public static string GetComputerName() {...}
}
```

```
public class NormalClass: CommonTask
{
    ...
}
```

 class StaticClassDemo.NormalClass

'NormalClass': cannot derive from static class 'CommonTask'

## Difference Between Static and Non-Static Class in C#

1. In C#, the static class is created by using the static keyword, the rest of the others are non-static classes.
2. We cannot create an instance of a static class even if reference variables are not possible. On the other hand, we can create both instance and reference variables using a non-static class.
3. We can access the members of a static class directly by using the class name. To access the non-static members, we need an instance or object of that class.
4. In static class, we can only define static members. On the other hand, inside a non-static class, we can define both static and non-static members.
5. A static class contains only a static constructor whereas a non-static class contains can both static and instance constructors.
6. Static classes are sealed and hence cannot inherit from another class. On the other hand, the non-static class can be inherited by another class.

In the next article, I am going to discuss the differences between [Variables, References, and Instances of a Class in C#](#) with Examples. Here, in this article, I try to explain **Static Class in C#** with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this Static Class in C# with Examples article.